Configure a Fixed SQL Server TCP Port for SST

Overview

On some Windows machines, SQL Server may be installed with **TCP/IP disabled**. Named SQL Server instances may also be configured to use a **dynamic TCP port** instead of a fixed port.

> **Terminology note:** SQL Server uses dynamic **ports**, not dynamic IP addresses. The server IP address is controlled by Windows and the network configuration.

A dynamic SQL port can change when the SQL Server service or computer restarts. This makes client-to-server connectivity unreliable because:

- a Windows Firewall rule created for the previous port may no longer match;
- client connection settings may continue pointing to the old port;
- virtual machines may connect successfully before a restart and fail afterward;
- troubleshooting becomes difficult because the SQL endpoint is not predictable; and
- SQL Server Browser or additional discovery configuration may otherwise be required for named instances.

For a stable SST client/server configuration, SQL Server should:

- have TCP/IP enabled;
- listen on a fixed TCP port;
- have dynamic SQL ports disabled;
- listen on all applicable server network interfaces; and
- have a matching inbound Windows Firewall rule.

The supplied scripts configure the SST SQL Server instance to use fixed TCP port **52525** and create an inbound firewall rule named **SST_Listening_Port**.

Environment Expected by the Scripts

The scripts use these defaults:

```
SQL instance:       SST
SQL TCP port:       52525
Firewall rule name: SST_Listening_Port
```

The SQL endpoint used by clients should be:

```
tcp:<server-name>,52525
```

Example:

```
tcp:DIR-VIRTUAL-SER,52525
```

Because a fixed TCP port is supplied, the SQL instance name does not need to be included in the client server address.

Files

The configuration package contains two PowerShell scripts:

```
Enable_SQL_TCP_52525.ps1
Create_SST_Listening_Port_Firewall_Rule.ps1
```

SQL configuration script

`Enable_SQL_TCP_52525.ps1` performs the following actions:

- loads the SQL Server WMI management provider installed by SQL Server Setup;
- locates the local SQL Server instance named `SST`;
- enables the SQL Server TCP/IP protocol;
- configures SQL Server to listen on all server IP interfaces;
- clears the dynamic TCP port setting;
- assigns fixed TCP port `52525`;
- restarts the `MSSQL$SST` SQL Server service; and
- verifies that SQL Server is listening on port `52525`.

Firewall configuration script

`Create_SST_Listening_Port_Firewall_Rule.ps1` performs the following actions:

- removes an existing Windows Firewall rule with the same display name, if one exists;
- creates an enabled inbound rule named `SST_Listening_Port`;
- allows inbound TCP connections on local port `52525`; and
- applies the rule to the Domain, Private, and Public firewall profiles.

Prerequisites

Before running the scripts, confirm the following:

- SQL Server is installed on the server.
- The SQL Server instance is named `SST`.
- Windows PowerShell 5.1 is available.
- The scripts are run from **Windows PowerShell as Administrator**.
- Port `52525` is not already assigned to another application.
- Restarting the SQL Server service is acceptable during the maintenance window.

The SQL configuration script uses the SQL Server WMI management components installed by SQL Server Setup. SQL Server Management Studio and the PowerShell `SqlServer` module are not required.

Part 1: Prepare the Script Files

Copy both scripts to the SQL Server machine. For example, place the extracted files in the Downloads folder.

Open **Windows PowerShell as Administrator** and go to the folder containing the scripts:

```
cd "$HOME\Downloads\SST-SQL-TCP-52525-Scripts"
```

Allow scripts only for the current PowerShell session:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force
```

Unblock the downloaded scripts:

```
Unblock-File ".\Enable_SQL_TCP_52525.ps1"
Unblock-File ".\Create_SST_Listening_Port_Firewall_Rule.ps1"
```

The execution-policy change applies only to the current PowerShell window. Closing the window restores the previous policy.

Part 2: Configure SQL Server TCP/IP and Port 52525

Run the SQL configuration script:

```
.\Enable_SQL_TCP_52525.ps1
```

The script restarts the SQL Server service so that the TCP/IP and fixed-port changes take effect.

A successful result should resemble:

```
Loading the SQL Server WMI management provider...
TCP/IP enabled for 'SST'.
Listen on all IPs enabled; dynamic ports cleared; fixed port set to 52525.
SUCCESS: SQL instance 'SST' is listening on TCP port 52525.

LocalAddress LocalPort State  OwningProcess
------------ --------- -----  -------------
::               52525 Listen         11108
0.0.0.0          52525 Listen         11108
```

The process ID will vary. These listener addresses are expected:

- `0.0.0.0:52525` means SQL Server is listening on all applicable IPv4 interfaces.
- `[::]:52525` means SQL Server is listening on all applicable IPv6 interfaces.

Optional: Configure without immediately restarting SQL Server

If a deployment tool or administrator will restart SQL Server later, run:

```
.\Enable_SQL_TCP_52525.ps1 -NoRestart
```

The new SQL listener will not become active until the SQL Server service is restarted.

To restart it later:

```
Restart-Service -Name 'MSSQL$SST' -Force
```

Part 3: Create the Windows Firewall Rule

Run the firewall script:

```
.\Create_SST_Listening_Port_Firewall_Rule.ps1
```

A successful result should report:

```
SUCCESS: Firewall rule 'SST_Listening_Port' allows inbound TCP port 52525.
```

Verify the rule:

```
Get-NetFirewallRule -DisplayName "SST_Listening_Port" |
    Get-NetFirewallPortFilter
```

The result should show:

```
Protocol  LocalPort  RemotePort
--------  ---------  ----------
TCP       52525      Any
```

Part 4: Validate the SQL Listener on the Server

Confirm that SQL Server is listening locally:

```
Get-NetTCPConnection -State Listen -LocalPort 52525
```

You may also use:

```
netstat -ano | findstr :52525
```

Expected listener entries include:

```
TCP    0.0.0.0:52525    0.0.0.0:0    LISTENING
TCP    [::]:52525       [::]:0       LISTENING
```

Confirm that the listening process belongs to SQL Server:

```
$connection = Get-NetTCPConnection -State Listen -LocalPort 52525 |
    Select-Object -First 1

Get-Process -Id $connection.OwningProcess
```

The process name should be:

```
sqlservr
```

Part 5: Validate Connectivity from a Client

On the client machine, open PowerShell and run:

```
Test-NetConnection <server-name> -Port 52525
```

Example:

```
Test-NetConnection DIR-VIRTUAL-SER -Port 52525
```

Expected result:

```
TcpTestSucceeded : True
```

This confirms that the client can establish a TCP connection to the SQL Server listener through the network and Windows Firewall.

> `Test-NetConnection` validates TCP reachability only. It does not validate SQL credentials, Windows Authentication, database permissions, encryption settings, or application configuration.

Part 6: Configure the Client SQL Endpoint

Use the server name and fixed port in the application or installer:

```
tcp:DIR-VIRTUAL-SER,52525
```

If the application does not accept the `tcp:` prefix, use:

```
DIR-VIRTUAL-SER,52525
```

Avoid using the following values from a separate client computer:

```
localhost\SST
127.0.0.1,52525
localhost,52525
```

Those values refer to the client computer itself, not the remote SQL Server.

When a fixed TCP port is specified, this form is usually unnecessary:

```
DIR-VIRTUAL-SER\SST,52525
```

The simpler host-and-port endpoint avoids dependence on SQL Server Browser and named-instance port discovery:

```
DIR-VIRTUAL-SER,52525
```

Troubleshooting

PowerShell reports that running scripts is disabled

Use a process-scoped execution-policy bypass:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force
```

Then rerun the script:

```
.\Enable_SQL_TCP_52525.ps1
```

The SQL instance is not found

List the locally installed SQL Server services:

```
Get-Service |
    Where-Object Name -like 'MSSQL*' |
    Select-Object Name, DisplayName, Status
```

If the instance name is not `SST`, pass the correct name to the script:

```
.\Enable_SQL_TCP_52525.ps1 -InstanceName "MyInstance"
```

For the default SQL Server instance, use:

```
.\Enable_SQL_TCP_52525.ps1 -InstanceName "MSSQLSERVER"
```

Port 52525 is already in use

Check which process is using the port:

```
Get-NetTCPConnection -LocalPort 52525 -ErrorAction SilentlyContinue |
    Select-Object LocalAddress, LocalPort, State, OwningProcess
```

Identify the process:

```
Get-Process -Id <OwningProcess>
```

Do not assign SQL Server to port `52525` until any port conflict has been resolved.

SQL Server restarts but no listener appears

Check the SQL Server service:

```
Get-Service -Name 'MSSQL$SST'
```

Check the SQL Server error log for TCP bind failures, port conflicts, or startup errors. The SQL Server error log is commonly located under the instance's `MSSQL\Log` folder.

Verify the configured registry values:

```
$instanceNames = Get-ItemProperty `
    'HKLM:\SOFTWARE\Microsoft\Microsoft SQL Server\Instance Names\SQL'

$instanceId = $instanceNames.SST
$tcpPath = "HKLM:\SOFTWARE\Microsoft\Microsoft SQL
Server\$instanceId\MSSQLServer\SuperSocketNetLib\Tcp"

Get-ItemProperty -Path $tcpPath
Get-ItemProperty -Path "$tcpPath\IPAll"
```

Expected values include:

```
Enabled             1
ListenOnAllIPs      1
TcpDynamicPorts     <blank>
TcpPort             52525
```

The server listener works, but the client connection fails

Confirm name resolution from the client:

```
Resolve-DnsName <server-name>
```

Confirm TCP connectivity:

```
Test-NetConnection <server-name> -Port 52525
```

If `TcpTestSucceeded` is `False`, investigate:

- the server's Windows Firewall rule;
- VMware, NSX, VLAN, or network access-control policies;
- endpoint security software;
- incorrect DNS resolution; or
- routing between the client and server networks.

If `TcpTestSucceeded` is `True` but SQL authentication fails, the network and firewall path are working. Investigate the SQL connection string, authentication type, SQL permissions, SPN or SSPI configuration, and application identity.

Operational Notes

- The SQL configuration script restarts the SQL Server service and temporarily interrupts active database connections.
- Run the SQL script during an approved maintenance window on shared or production servers.
- The firewall rule applies to all Windows Firewall profiles. Organizations with stricter security requirements may modify the rule scope or profiles.
- Restrict the firewall rule to approved client subnets when required by the organization's security policy.
- Port `52525` is a fixed application configuration choice, not a substitute for authentication or network security.
- Changing the SQL port requires corresponding updates to client connection settings and firewall rules.

Why This Configuration Works

A predictable SQL endpoint requires both a stable listener and an allowed network path.

The SQL configuration script establishes the stable listener:

```
SQL Server TCP/IP enabled
Dynamic TCP ports disabled
Fixed TCP port 52525 assigned
SQL Server listening on server network interfaces
```

The firewall script establishes the allowed inbound path:

```
Inbound TCP port 52525 allowed
Rule name SST_Listening_Port
All Windows Firewall profiles covered
```

Clients can then use a consistent SQL endpoint across SQL Server service restarts and Windows restarts:

```
tcp:<server-name>,52525
```

Script names

```
Enable_SQL_TCP_52525.ps1
Create_SST_Listening_Port_Firewall_Rule.ps1
```